

Büyük Dil Modellerinin İkilemi: Bilgili ama Bazen Hayalperest

LLM'ler engin parametrik bilgiye sahiptir, ancak bu bilgi statik, güncel olmayabilir ve halüsinasyonlara yol açabilir.

LLM'leri, genel hukuk anlayışı çok geniş ancak davaya özel kanun kitaplığına erişimi olmayan bir hakime benzetebiliriz. Karar verebilir, ancak en doğru ve kanıta dayalı kararı vermek için harici kaynaklara başvurması gerekir. Bu durum, LLM'lerin en büyük zorluklarıdır:

- Bilgi Kesintisi (Knowledge Cutoff): Eğitim verilerinin ötesindeki güncel olayları bilemezler.
- Halüsinasyon: Bilgi boşluklarını, kulağa makul gelen ancak yanlış olan bilgilerle doldurma eğilimindedirler.
- Kaynak Belirsizliği: Verdikleri bilginin kaynağını belirtemezler, bu da güvenilirliği azaltır.



Çözüm Sahneye Çıkıyor: Retrieval-Augmented Generation (RAG)

RAG, LLM'lerin bu sınırlamalarını aşmak için iki güçlü yeteneği birleştiren bir mimaridir: **Bilgi Erişimi (Retrieval)** ve **Dil Üretimi (Generation)**. Modelin yeniden eğitilmesine gerek kalmadan, harici ve güncel bilgi kaynaklarına erişmesini sağlar.



Retrieval (Erişim)

Belirli bir sorguya yanıt olarak, geniş bir veri tabanından (PDF'ler, web sayfaları, veritabanları) en ilgili bilgi parçacıklarını bulup getirme sürecidir. Anlamsal arama tekniklerini kullanır.



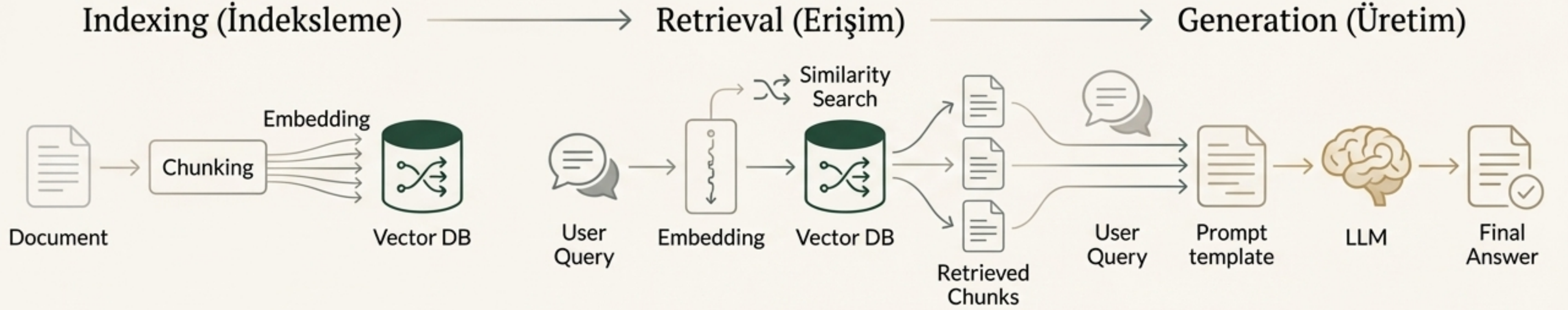
Generation (Üretim)

'Erişilen' bu harici bilgiyi bağlam olarak kullanarak, kullanıcıya tutarlı, doğru ve kanıta dayalı bir yanıt oluşturan LLM'in kendisidir.

Kısacası RAG, LLM'e 'cevap vermeden önce bu belgelere göz at' diyerek ona bir araştırma yeteneği kazandırır.

Temel RAG Mimarisi: Adım Adım İşleyiş

En basit haliyle bir RAG sistemi, 'Naive RAG' olarak da bilinen, doğrusal bir iş akışını takip eder. Bu temel yapı, RAG'ın evrimini anlamak için bir başlangıç noktasıdır.



Harici belgeler yüklenir, daha küçük parçalara ('chunk') ayrılır, her bir parça sayısal bir vektöre ('embedding') dönüştürülür ve bir Vektör Veritabanında saklanır.

Kullanıcının sorusu da bir vektöre dönüştürülür. Bu sorgu vektörü, veritabanındaki en benzer (en ilgili) belge parçacıklarını bulmak için kullanılır.

En ilgili belge parçacıkları, orijinal soruyla birlikte bir prompt'a eklenir ve LLM'e gönderilir. LLM, bu zenginleştirilmiş bağlamı kullanarak yanıtını üretir.

Not: Bu yaklaşım güçlü bir başlangıç olsa da, erişilen bilginin kalitesi ve ilgililiği konusunda zorluklar yaşayabilir (düşük hassasiyet, düşük geri çağırma).

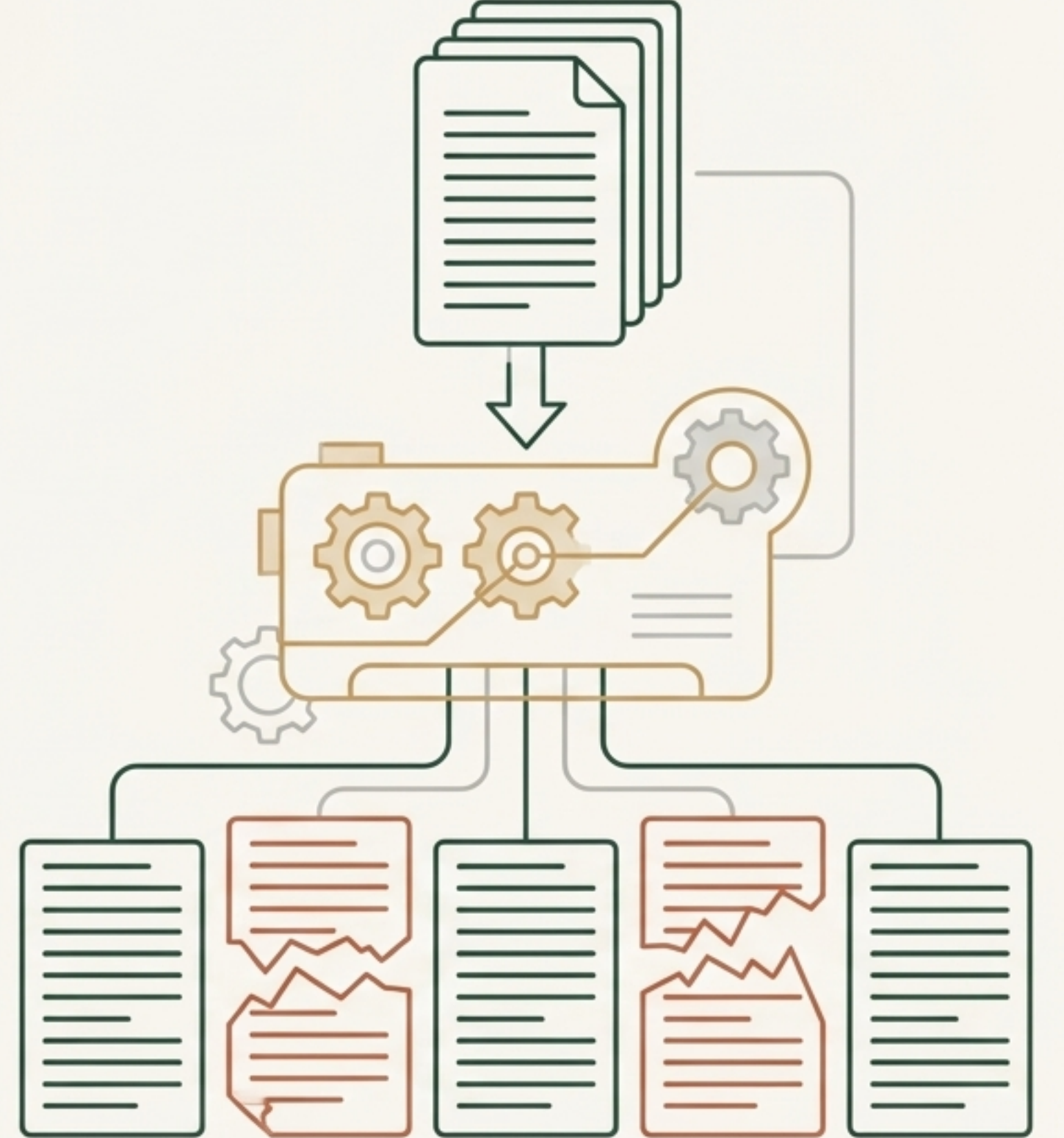
RAG Performansının Temeli: Etkili 'Chunking' (Parçalama) Sanatı

Bir RAG sisteminin kalitesi, getirdiği bilginin kalitesine bağlıdır. Bu sürecin en kritik adımı ise **Chunking'dir**: Büyük belgeleri, anlamsal bütünlüğü koruyarak daha küçük, yönetilebilir ve aranabilir parçalara ayırma işlemidir.

Neden Kritik?

- **Bağlam Kaybı**: Çok küçük parçalar, cümlenin veya paragrafın anlamını yitirmesine neden olabilir.
- **Bağlamsal Gürültü**: Çok büyük parçalar, soruyla alakasız bilgiler içererek LLM'i yanıltabilir.
- **Verimlilik**: Doğru boyuttaki parçalar, arama sürecini hızlandırır ve maliyetleri düşürür.

'Doğru chunking stratejisi, projenizin başarısını doğrudan etkiler. Şimdi en yaygın ve gelişmiş teknikleri inceleyelim.'



Chunking Teknikleri 1: Temel Yaklaşımlar

Fixed-Size Chunking (Sabit Boyutlu)

En basit yöntemdir. Metni, anlamsal yapıyı dikkate almadan belirli bir karakter sayısına göre böler. Hızlıdır ancak cümleleri veya paragrafları ortadan bölebilir.

Parametreler: ``chunk_size`` (parça boyutu), ``chunk_overlap`` (bağlamı korumak için parçalar arası karakter örtüşmesi).

Örnek Metin

"Artificial Intelligence is transforming industries. It has applications in healthcare, finance, education, and more."

Sonuç (50 karakter, 10 örtüşme)

'Artificial Intelligence is transforming industries.

ng industries. It has applications in healthcare,

ions in healthcare, finance, education, and mo'

Recursive Chunking (Özyineli)

Metnin doğal yapısını korumaya çalışır. Paragraflar (``\n\n``), satırlar (``\n``), boşluklar gibi bir ayırıcı hiyerarşisi kullanarak metni akıllıca böler.

Mantık: Önce paragraflara bölmeyi dener. Parçalar hala çok büyükse, bir sonraki ayırıcıyla (cümleler) tekrar böler. Bu, anlamsal bütünlüğü en üst düzeye çıkarır.

Örnek Metin

'### Introduction\n\nArtificial Intelligence is growing rapidly.\n\n### Applications\n\nAI is used in healthcare and education.'

Sonuç (Ayırıcı: ``\n\n``)

Introduction
n\nArtificial Intelligence is growing rapidly.

Applications
n\nAI is used in healthcare and education.

Chunking Teknikleri 2: Yapısal ve Multimodal Belgeler

Gerçek dünya verileri sadece düz metinlerden oluşmaz. Document-based chunking, belgenin kendi formatını (başlıklar, kod blokları, tablolar) birer ayırıcı olarak kullanır.

Markdown

Strategy: `#` ve `##` gibi başlık etiketlerine göre bölerek belgenin mantıksal akışını korur.

```
# Giriş
Bu bir giriş paragrafıdır.

## Alt Başlık
Bu bir alt başlıktır.
```



```
# Giriş
Bu bir giriş paragrafıdır.

## Alt Başlık
Bu bir alt başlıktır.
```

Python Kodu

Strategy: `class` ve `def` gibi anahtar kelimelere göre bölerek fonksiyonları ve sınıfları bütün olarak tutar. `RecursiveCharacterTextSplitter.from\_language("python")` gibi araçlar kullanılır.

```
class VeriIsleyici:
    def __init__(self):
        pass

def veri_yukle(dosya_yolu):
    print(f"Yükleniyor: {dosya_yolu}")
```



```
class VeriIsleyici:
    def __init__(self):
        pass

def veri_yukle(dosya_yolu):
    print(f"Yükleniyor: {dosya_yolu}")
```

Tablolar

Strategy: Tablo yapısını bozmamak için HTML (<table>) veya CSV formatına dönüştürülür. Daha iyi anlamsal arama için, tablonun bir özetini çıkarıp bu özetini embed etmek daha etkilidir.

İsim	Rol
Ali	Mühendis
Ayşe	Tasarımcı

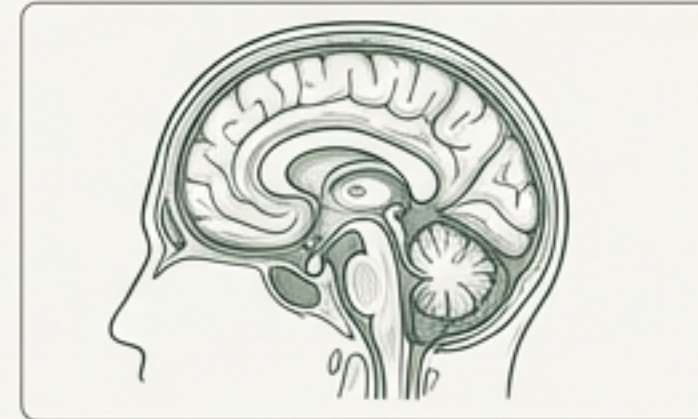


```
<table>
  <thead>
    <tr><th>İsim</th><th>Rol</th></tr>
  </thead>
  <tbody>
    <tr><td>Ali</td><td>Mühendis</td></tr>
    <tr><td>Ayşe</td><td>Tasarımcı</td></tr>
  </tbody>
</table>
```

'Bu tablo çalışanları ve rollerini listeler.'

Görseller (Multimodal)

Görseller, GPT-4 Vision gibi multimodal modeller kullanılarak metinsel bir özete dönüştürülür. Bu özet metin embed edilir. `unstructured.io` gibi kütüphaneler bu işlemi kolaylaştırır.



'Bu görüntü, sağlıklı bir insan beyninin sagittal bir görünümünü göstermektedir.'

Chunking Teknikleri 3: Anlamsal ve Agentic Yaklaşımlar

En gelişmiş RAG sistemleri, metni anlam düzeyinde ele alan chunking stratejileri kullanır. Bu yöntemler hesaplama olarak daha yoğundur ancak erişim kalitesini önemli ölçüde artırır.

1. Semantic Chunking (Anlamsal Parçalama)

Nasıl Çalışır?

Metni sabit kurallara göre değil, cümleler arasındaki anlamsal benzerliğe göre böler. Embedding modelleri kullanarak, birbiriyle anlamsal olarak en ilişkili cümle gruplarını bir arada tutar. Bir konudan diğerine geçiş anları, doğal bölünme noktaları olur.

Araç: LlamaIndex'in `SemanticSplitterNodeParser`'ı bu tekniği uygular.

Örnek Metin

"AI is the simulation of human intelligence by machines. It has applications across domains. For instance, in healthcare, AI is used for diagnosis. In finance, it's used for fraud detection."

Genel Tanım

Sağlık Uygulaması

Finans Uygulaması

2. Agentic Chunking (Agent Tabanlı Parçalama)

Nasıl Çalışır?

Chunking kararını bir LLM'in kendisine yaptıran en ileri tekniktir. Metin önce 'önermeler' (atomik, kendi kendine yeten ifadeler) haline getirilir. Ardından bir LLM ajanı bu önermeleri değerlendirir ve mantıksal devamlılığa göre hangi önermelerin aynı chunk'ta yer alacağına karar verir.

Örnek Önermeler

AI üretkenliği artırır.

AI görevleri otomatikleştirir.

AI etik kaygılar doğurur.

AI sistemlerindeki yanlışlık önemli bir sorundur.

LLM Ajan Kararı



AI üretkenliği artırır. AI görevleri otomatikleştirir.

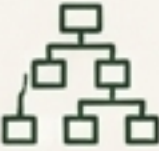
Faydalar

AI etik kaygılar doğurur. AI sistemlerindeki yanlışlık önemli bir sorundur.

Zorluklar

Hangi Chunking Stratejisi Sizin İçin Doğru?

Mükemmel bir 'tek beden' chunking stratejisi yoktur. Seçim; belgenizin yapısına, arama hassasiyetinize ve hesaplama bütçenize bağlıdır.

Strateji	Temel Mantık	Hız & Karmaşıklık	Anlamsal Bütünlük	İdeal Kullanım Alanı
 Fixed-Size	Sabit karakter sayısı	Çok Hızlı / Düşük	Düşük (Cümleleri bölebilir)	Yapısal olmayan, kısa metinler, hızlı prototipleme.
 Recursive	Öncelikli ayırıcı listesi	Hızlı / Orta	Orta-Yüksek (Yapıyı korur)	Yarı yapılandırılmış metinler (makaleler, raporlar). En dengeli yaklaşım.
 Document-Based	Belge formatı (Başlık, Kod)	Hızlı / Orta	Yüksek (Mantıksal akışı korur)	İyi yapılandırılmış belgeler: Markdown, kod dosyaları, yasal metinler.
 Semantic	Embedding benzerliği	Yavaş / Yüksek	Çok Yüksek (Konu bazlı gruplar)	Anlamsal nüansın kritik olduğu akademik makaleler, tıbbi kayıtlar.
 Agentic	LLM tabanlı karar verme	Çok Yavaş / Çok Yüksek	Üstün (İnsan benzeri gruplama)	Karmaşık, bilgi-yoğun analiz gerektiren alanlar (hukuk, politika).

Lorem ipsum dolor sit amet, consectetur adipiscing elit, kökeni ve bağlamı hakkında bilgi veren metadata eklemek, RAG sisteminizi bir üst seviyeye taşır. Metadata, her chunk'a yapıştırılmış aligumr au en. consectetur adipiscing elit, kökeni ve bağlamı hakkında bilgi veren metadata eklemek, RAG sisteminizi bir üst seviyeye taşır. Metadata, her chunk'a yapıştırılmış aligumr au en. consectetur adipiscing elit, kökeni ve bağlamı hakkında bilgi veren metadata eklemek, RAG sisteminizi bir üst seviyeye taşır. Metadata, her chunk'a yapıştırılmış aligumr au en.

document_name: report.pdf

page_number: 12

chunk_number: 3

content_type: Paragraph

date: 2023-10-26

Parçacıklara Kimlik Kazandırmak: Metadata'nın Gücü

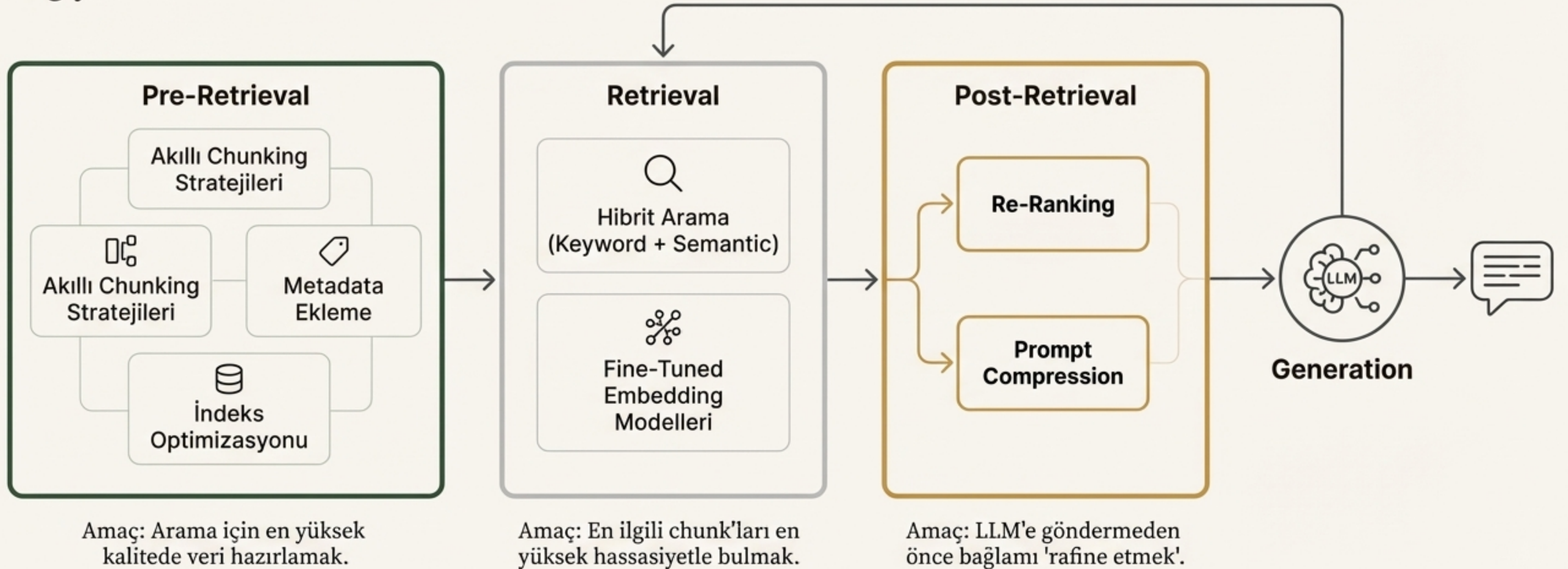
Chunking, sadece metni bölmek değildir. Her bir parçaya, onun kökeni ve bağlamı hakkında bilgi veren **metadata** eklemek, RAG sisteminizi bir üst seviyeye taşır. Metadata, her chunk'a yapıştırılmış akıllı bir kimlik etiketidir.

Metadata Neden Hayati?

- **Filtreleme:** 'Sadece 'John Doe' tarafından yazılmış ve 2023'ten sonraki tablolardan bilgi getir' gibi karmaşık sorguları mümkün kılar.
- **Kaynak İzleme:** LLM bir yanıt ürettiğinde, bilginin orijinal belgenin tam olarak neresinden geldiğini kullanıcıya gösterebilir, şeffaflığı artırır.
- **Alaka Düzeyini Artırma:** Arama sonuçlarını sıralarken metadata kullanılabilir. Örneğin, daha yeni tarihli chunklara öncelik verilebilir.

Evrimin Tamamlanması: Gelişmiş RAG (Advanced RAG) Mimarisi

Naive RAG'ın sınırlamalarını aşmak için, Gelişmiş RAG mimarisi hem erişim öncesi hem de erişim sonrası süreçlere optimizasyonlar ekler. Amaç, LLM'e mümkün olan en temiz, en ilgili ve en yoğun bilgiyi sunmaktır.



Teoriden Pratięe: Basit Bir RAG Sisteminin Anatomisi

Gelişmiş vektör veritabanları olmadan da temel bir RAG sistemi kurulabilir. Aşağıdaki örnek, anahtar kelime tabanlı bir erişim mekanizması kullanarak bu mimarinin temel mantığını göstermektedir.

Örnek Sistem Adımları

- Document Loading:** Kullanıcı bir PDF, DOCX veya TXT dosyası yükler. `PyPDFLoader` gibi kütüphanelerle ham metin okunur.
- Text Chunking (Dolaylı):** Metin, `\n\n` ile paragraflara ayrılır. Bu, basit bir recursive chunking formudur. Çok kısa paragraflar filtrelenir.
- Scoring & Retrieval:**
 - Yöntem:** Anahtar kelime tabanlı arama. Kullanıcının sorusundaki kelimeler, her paragrafta aranır.
 - Puanlama:** Bir paragrafta ne kadar çok anahtar kelime geçerse, puanı o kadar yüksek olur.
 - Erişim:** En yüksek puanlı ilk 5 paragraf 'en ilgili bağlam' olarak seçilir.
- Context Injection:**
 - Yöntem:** Seçilen en ilgili paragraflar, sistem prompt'una şu şekilde eklenir: "Aşağıdaki dosya içeriğine dayanarak cevap ver: {relevant_context}"
- LLM Generation:** LLM, bu zenginleştirilmiş prompt'u alarak dosyaya özel ve bağlama dayalı bir cevap üretir.

RAG Bölümü	Kavramlar	Parametreler
Loading	PyPDFLoader`, `Docx2txtLoader	temp_path
Chunking	Paragraf bölme (`\n\n`)	max_chars=2000 `top_k=5
Retrieval	Keyword-match, relevance sort	keywords, `score
Injection	`system_prompt` genişletme	relevant_context
Generation	API çağrısı (Groq, OpenAI)	`temperature`, `max_tokens`

Stratejik Karar Anı: RAG mı, Fine-Tuning mi?

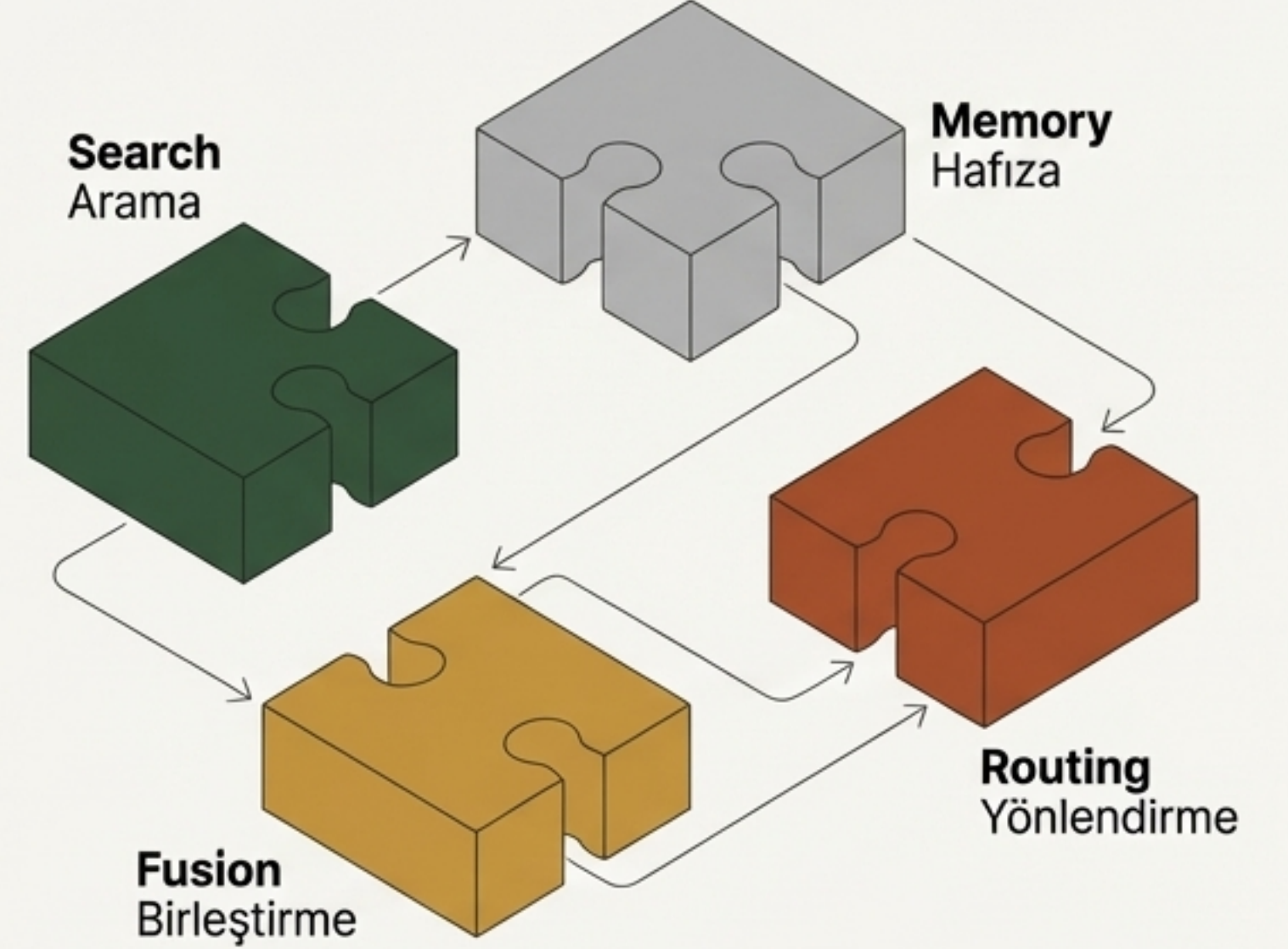
Bu iki teknik birbirinin rakibi değil, farklı sorunları çözen tamamlayıcı yaklaşımlardır. Doğru olanı seçmek, projenizin hedeflerine bağlıdır. Bazen en iyi çözüm ikisini bir arada kullanmaktır.



Özellik	RAG (Retrieval-Augmented Generation)	Fine-Tuning (İnce Ayar)
Temel Amaç	Harici, dinamik bilgiye erişim sağlamak.	Modelin stilini, tonunu veya davranışını özelleştirmek.
Bilgi Güncelleme	Kolay ve anlık. Yeni belge eklemek yeterlidir.	Zor ve maliyetli. Modelin yeniden eğitilmesi gerekir.
Halüsinasyon Riski	Düşük. Cevaplar kanıta dayalıdır.	Orta. Model hala 'uydurabilir'.
Kaynak Gösterme	Kolay. Erişilen chunk'lar kaynak olarak gösterilebilir.	İmkansız. Bilgi parametrik bellektedir.
Maliyet (Uygulama)	Daha düşük başlangıç maliyeti. Vektör DB ve API kullanımı.	Yüksek. Etiketli veri seti hazırlama ve eğitim süreci gerektirir.
Ne Zaman Kullanılır?	Bilgi-yoğun görevler: Soru-cevap, güncel olaylar, doküman analizi.	Stil ve format adaptasyonu: Belirli bir tonda yazan chatbot, kod üretimi.

Geleceğin Mimarisi: Modüler RAG ve Ufuktaki Yenilikler

RAG'ın evrimi, sabit boru hatlarından esnek ve birleştirilebilir modüler yapılara doğru ilerliyor. Modüler RAG, arama, hafıza, yönlendirme ve birleştirme gibi işlevsel modüllerin Lego parçaları gibi bir araya getirildiği bir yaklaşımdır. Bu, her göreve özel, son derece optimize edilmiş sistemler oluşturmamızı sağlar.



Gelecek Zorluklar ve Fırsatlar

- **Uzun Bağlam Pencereleri (Long Context Windows):** LLM'lerin bağlam pencereleri büyüdükçe, RAG'ın rolü "iğneyi bulmaktan" ziyade "samanlıktaki en önemli iğneleri bulup öne çıkarmaya" evrilecektir.
- **Multimodal RAG:** Metin, görsel, ses ve video verilerini bir arada işleyebilen, daha bütünsel RAG sistemleri yaygınlaşacak.
- **Production-Ready RAG:** Performans, verimlilik, veri güvenliği ve gizlilik gibi mühendislik zorluklarının üstesinden gelmek, üretim ortamları için kritik öneme sahip olacaktır.
- **Daha Akıllı Erişim (Smarter Retrieval):** Sorguyu alt sorgulara bölme (sub-queries), yinelemeli erişim (recursive retrieval) ve LLM'in ne zaman erişim yapacağına kendisinin karar verdiği (adaptive retrieval) teknikler standartlaşacak.